FICHES MÉMO

Carnet de l'apprenant

Une fiche mémo par séance : l'essentiel à retenir, les commandes, un prompt type et le travail à préparer.

PRÉPARÉ POUR

KOUA BLAISE JUNIOR ELISE

Développer des applications web avec Claude Code · Projet Faya

S1

Prise en main

Ce que j'ai appris

- Le **terminal** : une fenêtre où je tape des commandes. Je lis toujours ce qui s'affiche.
- **Git** : l'historique de mon projet. 3 choses pour l'instant : dépôt, `git status`, commit (= photo sauvegardée).
- **Claude Code** : il ne fait pas que discuter, il agit (lit mes fichiers, en crée, en modifie).
- Mon **CONTEXT.md** et mon **CLAUDE.md** disent à Claude qui je suis et comment m'aider.

Commandes à retenir

```
cd <dossier>      # se déplacer
ls                # lister ce qu'il y a ici
pwd               # où suis-je ?
git status        # où j'en suis côté sauvegarde
git add -A        # préparer tout à sauvegarder
git commit -m "message"   # sauvegarder une photo
/prime            # Claude charge mon contexte
```

Prompt type

> « Explique-moi ce que contient ce workspace et à quoi sert chaque dossier. »

Réflexe quota (Pro)

Rester sur **Sonnet**, faire des sessions courtes.

Mon devoir

Réfléchir à l'app que je veux construire : quel problème réel, pour qui, 3 fonctions maximum. L'écrire en 5 lignes.

S2

La méthode de travail

Ce que j'ai appris

- **Ne jamais coder tout de suite.** D'abord comprendre, puis écrire ce que je veux, puis découper, puis exécuter.
- La chaîne : **brainstorming** → **spec** → **plan** → **exécution**.
- Une **spec** = le plan de mon app : objectif, ce que je fais, ce que je ne fais PAS (important !), les étapes.
- **YAGNI** : je garde 3 fonctions max pour commencer. J'ajouterai après.

Comment je lance ça

> « Je veux construire [mon idée]. Aide-moi à la cadrer, pose-moi les questions une par une, puis propose-moi 2-3 approches avant d'écrire une spec. »

Claude pose UNE question à la fois : je réponds tranquillement.

Réflexe quota (Pro)

Réfléchir (brainstorming) coûte peu. Coder à l'aveugle coûte cher. Je cadre d'abord.

Mon devoir

Relire ma spec à tête reposée, noter 3 choses à ajouter ou retirer.

S3

Continuité parfaite

Ce que j'ai appris

- Une conversation Claude a une **limite**. Quand elle sature, je perds le contexte.
- La règle d'or : **tout ce qui a de la valeur va dans un fichier**, pas dans la conversation.
- Les 4 fichiers d'un projet :
- **CLAUDE.md** — la constitution (stack, conventions, objectifs)
- **PROGRESS.md** — où j'en suis
- **DECISIONS.md** — pourquoi j'ai fait tel choix
- **NEXT_SESSION.md** — le prompt pour repartir dans une conversation neuve
- `/nouveau-projet` crée tout ça AVANT de coder.
- `/reprendre` relit tout et me remet dans le bain en 30 secondes.

Commandes à retenir

```
/nouveau-projet   # initialise un projet (structure + plan avant code)
/reprendre        # reprend un projet là où je l'avais laissé
npm run dev       # lance mon app en local pour la voir dans le navigateur
```

Réflexe quota (Pro) — LE plus important

Quand je reviens, je **n'allonge pas** une vieille conversation : j'en ouvre une neuve et je tape `/reprendre`. Beaucoup moins cher, plus rapide.

Mon devoir

Demain, ouvrir une NOUVELLE conversation et reprendre mon projet seul avec `/reprendre`. Vérifier que tout repart.

S4

Lire et piloter du code

Ce que j'ai appris

- Un prompt flou donne un résultat flou. Les **4 ingrédients d'un bon prompt** : 1. **Contexte** — sur quoi je travaille 2. **Objectif** — ce que je veux 3. **Contraintes** — comment, avec quels outils 4. **Résultat attendu** — à quoi ça ressemble si c'est réussi
- Je **lis ce que Claude fait** (quels fichiers il touche). Je ne valide pas en aveugle.
- Si je ne comprends pas, je demande à Claude d'expliquer avant d'accepter.
- Je construis **par petites touches**, je vérifie dans le navigateur à chaque fois.
- Face à une erreur : je **copie le message** et je le donne à Claude.

Prompt type (4 ingrédients)

> « Dans mon app Next.js (contexte), crée une page de connexion (objectif), avec Tailwind et un champ email + mot de passe (contraintes). Je veux voir un formulaire centré qui s'affiche sans erreur (résultat attendu). »

Réflexe quota (Pro)

Petites touches claires = moins cher qu'un gros prompt confus refait 3 fois.

Mon devoir

Ajouter seul une petite page ou un élément à mon app, avec un bon prompt. Noter une question si je bloque.

S5

C'est quoi tout ça ?

Analogie : **Claude Code = un employé très doué**. Voici ses outils et ses règles.

Brique	C'est quoi	Analogie
Commande (`/prime`)	Un raccourci que je tape	Une macro
Skill	Une méthode de travail qu'il applique	Un mode d'emploi
Agent	Un collègue spécialiste à qui il délègue	Un expert qu'on appelle
MCP	Une prise vers un service extérieur (base, GitHub, navigateur)	Un câble USB vers le monde réel
Hook	Une règle automatique (ex. bloquer une clé)	Un garde-fou
Plugin	Un pack qui apporte skills+agents+commandes+MCP	Une extension à installer

Où ça vit dans mon Jarvis

```
.claude/commands/   → mes commandes
.claude/skills/     → mes skills
.claude/agents/     → mes agents
.claude/hooks/      → mes garde-fous
.mcp.json           → mes prises MCP
PLUGINS.md          → les plugins à installer
```

Réflexe quota (Pro)

Déclencher un **agent** = une 2e conversation = ça consomme. Je l'utilise quand ça vaut vraiment le coup.

Mon devoir

Écrire une phrase perso pour chaque brique, avec mes propres mots.

S6

Design avec Claude Design, puis code

La méthode : design d'abord, code ensuite

Je ne code PAS l'interface à l'aveugle. Je la **dessine d'abord dans Claude Design**, je valide le visuel, PUIS je la transpose en code.

Ce que j'ai appris

- **Claude Design** sert à maquetter l'interface rapidement. J'itère sur le visuel là, pas dans le code.
- Je décris l'ambiance (3 adjectifs), une référence (« comme Stripe »), les couleurs, la disposition. Claude ne devine pas mon goût.
- Une fois la maquette validée, **Claude Code la transpose** en Next.js + Tailwind.
- Les skills qui aident à transposer fidèlement : `frontend-design`, `ui-design-system`, `tailwind-patterns`, `theme-factory`.
- Je garde UNE ambiance cohérente sur toutes les pages.

Le cycle

1. Décrire → **maquetter dans Claude Design**.
2. Ajuster le visuel (couleurs, disposition) jusqu'à ce que ça me plaise.
3. Demander à Claude Code de **transposer la maquette en code** Next.js + Tailwind.
4. Comparer le rendu codé à la maquette, corriger les écarts.

Prompt type (transposition)

> « Voici ma maquette Claude Design pour la page d'accueil. Transpose-la en page Next.js + Tailwind, fidèle au visuel, en appliquant la skill frontend-design. »

Réflexe quota (Pro)

Itérer le visuel dans Claude Design coûte moins que refaire du code 5 fois.

Mon devoir

Maquetter seul une page secondaire dans Claude Design (même ambiance), sans encore la coder.

S7

Responsive et accessibilité

Ce que j'ai appris

- La plupart des gens utilisent mon app sur **mobile**. Si elle casse sur petit écran, elle est inutilisable.
- **Responsive** : Claude adapte avec les classes Tailwind (`sm:`, `md:`, `lg:`). Je décris les écrans cibles.
- **Accessibilité (a11y)** : contrastes suffisants, textes alternatifs, navigation au clavier, labels de formulaire. C'est de la qualité et ça n'exclut personne.
- L'agent `web-accessibility-checker` (ou la skill `accessibility`) audite une page et me dit quoi corriger.
- Je teste en **vue mobile** : F12 → icône mobile dans les DevTools.

Prompt type

> « Teste ma page en vue mobile et corrige ce qui casse sur petit écran. Puis fais auditer l'accessibilité et applique les corrections prioritaires (contraste, labels, navigation clavier). »

Réflexe quota (Pro)

Je fais auditer par un agent surtout les pages importantes, pas toutes.

Mon devoir

Tester toute l'app sur mon téléphone (ou en vue mobile) et noter 3 problèmes.

S8

Commandes et skills perso

Ce que j'ai appris

- Je peux **fabriquer mes propres raccourcis**. Une commande = un fichier `.md` dans `.claude/commands/`. Son contenu est l'instruction exécutée quand je tape `/nom`.
- Une **commande** = MON raccourci perso. Une **skill** = une méthode fournie que j'invoque à la demande.
- Exemples de commandes utiles : `/point` (état de l'app), `/explique` (explique un fichier simplement), `/todo` (ce qu'il reste).

Créer une commande

1. Créer `.claude/commands/point.md`
2. Y écrire l'instruction, ex. :

   > « Résume l'état actuel de mon app en lisant PROGRESS.md, puis donne-moi les 3 prochaines actions. »

3. Taper `/point` pour la lancer.

Réflexe quota (Pro)

Une bonne commande **standardise mes prompts** → moins d'allers-retours, moins de quota gaspillé.

Mon devoir

Imaginer une 3e commande qui me faciliterait la vie et l'écrire.

S9

MCP en pratique

Ce que j'ai appris

- Un **MCP** branche Claude sur un service réel. Aujourd'hui : **Supabase** (base de données + auth, gratuit) et le **navigateur**.
- Mes clés ne vont **JAMAIS** dans le chat ni en dur dans le code. Elles vivent dans `.env` (ignoré par git) et dans le coffre `secrets/`.
- Workflow secrets : `/ajouter-cle` pour stocker, `/generer-env` pour injecter dans un projet.
- Je vérifie les connexions MCP avec `/mcp`.

Étapes

1. Créer un projet Supabase (gratuit).
2. Récupérer le token (Dashboard > Account > Access Tokens).
3. `/ajouter-cle` pour le stocker (jamais en clair dans le chat).
4. Le mettre dans `.env`, relancer Claude Code.
5. `/mcp` → vérifier que Supabase est connecté.

Pièges

- Supabase gratuit se met en **pause après 7 jours** sans activité → erreur « fetch failed ». Solution : réveiller le projet dans le dashboard.

Réflexe quota (Pro)

Les requêtes via MCP consomment un peu : rester sobre.

Mon devoir

Lister les 2-3 tables dont mon app aura besoin.

S10

Base de données

Ce que j'ai appris

- Une **table** = un tableur (lignes = enregistrements, colonnes = champs).
- Une **relation** = un lien entre tables (une tâche appartient à un utilisateur).
- **CRUD** = Créer, Lire, Mettre à jour, Supprimer. C'est 90% d'une app.
- La commande `/design-database-schema` aide à concevoir proprement.
- Je note mon schéma dans **DECISIONS.md** (c'est un choix structurant).

Prompt type

> « Aide-moi à concevoir le schéma de mon app : [décrire les entités]. Puis crée la table X dans Supabase et une page qui liste ses lignes. »

Pièges

- Supabase gratuit : une lecture peut être **tronquée à 1000 lignes** sans erreur. Pas grave à mon échelle.

Réflexe quota (Pro)

Faire concevoir tout le schéma d'un coup, pas table par table sur 10 messages.

Mon devoir

Ajouter la mise à jour OU la suppression (U ou D du CRUD) sur une entité, seul.

S11

Authentification

Ce que j'ai appris

- **Authentification** = prouver qui je suis (login). **Autorisation** = ce que j'ai le droit de faire.
- **Supabase Auth** fournit tout clé en main (email/mot de passe). Je ne réinvente pas.
- Une **session** = l'app se souvient de moi après connexion.
- Une **route protégée** = une page visible seulement si je suis connecté.
- Chaque utilisateur ne doit voir **que ses données** (règles d'accès Supabase, RLS).

Prompt type

> « Branche l'authentification Supabase (email/mot de passe) : crée les pages inscription et connexion, protège la page X (redirige si non connecté), et fais que chaque utilisateur ne voie que ses propres données. »

Parcours à tester

Créer un compte → se connecter → voir ses données → se déconnecter.

Réflexe quota (Pro)

L'auth est un bloc connu : un prompt bien cadré suffit, j'évite de tâtonner.

Mon devoir

Tester les cas limites (mauvais mot de passe, email déjà pris) et noter le comportement.

S12

Économie Pro et continuité avancée

Ce que j'ai appris — les leviers pour tenir mon quota Pro

1. **Modèle** : rester sur **Sonnet** par défaut, réserver les modèles lourds aux moments durs.
2. `/clear` entre deux tâches sans rapport : repartir propre.
3. **Sessions courtes et ciblées** : une tâche = une session.
4. **Continuité par fichiers** : reprendre neuf avec `/reprendre` plutôt qu'allonger.
5. **Prompts précis** (les 4 ingrédients de S4) : moins d'allers-retours.
6. **Agents avec parcimonie** : chaque agent = une conversation en plus.

Le réflexe-clé

Une conversation qui traîne devient lente et chère. J'ouvre une **conversation neuve** et je tape `/reprendre`.

Le checkpoint automatique

Vers ~70% de la fenêtre, le Jarvis fait un checkpoint tout seul (PROGRESS + NEXT_SESSION + INDEX). Je peux alors fermer et reprendre neuf.

Mon devoir

Tenir un mini-journal de ma consommation sur 2 jours (quand ça consomme le plus) et le rapporter.

S13

Agents et revue de code

Ce que j'ai appris

- Un **agent** = un collègue spécialiste à qui Claude délègue.
- `code-reviewer` : relit mon code (bugs, sécurité, qualité).
- `debugger` + skill `systematic-debugging` : pour un bug, je ne tâtonne pas.
- La commande `/code-review` fait une revue cadrée.
- **Debug systématique** : reproduire → formuler une hypothèse → tester → isoler → corriger → vérifier. Jamais « j'essaie au hasard ».
- Face aux retours : je **trie** (bugs réels d'abord), je ne corrige pas tout aveuglément.

Prompt type

> « Lance une revue de code (/code-review) sur ce fichier. Classe les problèmes par priorité et explique chacun avant de corriger. »

Réflexe quota (Pro)

Une revue = une grosse conversation. Je la lance sur des morceaux ciblés.

Mon devoir

Relancer une revue seul après mes corrections et vérifier que les points sont réglés.

S14

Hooks et garde-fous

Ce que j'ai appris

- Un **hook** = une règle automatique déclenchée à un moment. Mon Jarvis en a déjà :
- **dangerous-command-blocker** : bloque les commandes destructrices.
- **secret-scanner** : m'empêche de committer une clé.
- **blocage .env** en écriture : protège mes secrets.
- **formatage auto** (prettier/black) après édition : code propre sans y penser.
- **scope-guard** : me prévient si je modifie des fichiers hors du périmètre prévu.
- Ces garde-fous rattrapent les erreurs classiques de débutant.
- Mais la **discipline reste la base** : clés dans `.env`, jamais dans le chat ni en dur.

Vérifs utiles

```
git status        # mon .env ne doit PAS apparaître (il est gitignored)
```

En cas de blocage légitime

Si un hook bloque une commande correcte (faux positif), je **reformule** plutôt que de désactiver le garde-fou.

Réflexe quota (Pro)

Les hooks sont locaux : ils ne consomment pas de quota Claude.

Mon devoir

Relire mon historique git et vérifier qu'aucune clé n'a jamais été commitée.

S15

Déploiement

Ce que j'ai appris

- En local (`npm run dev`), seul moi vois l'app. **Déployer** = la rendre accessible par une URL publique.
- **Vercel** : hébergement fait pour Next.js, gratuit pour démarrer, se connecte à mon repo GitHub.
- Mes **variables d'environnement** doivent être recopiées dans les réglages Vercel (jamais dans le code, jamais commitées). L'app en prod lit celles de Vercel.
- **Preview** : chaque changement crée une URL de test avant la prod.

Étapes

1. Pousser le code sur GitHub (repo privé).
2. Importer le projet dans Vercel.
3. Reporter mes variables d'env dans les réglages Vercel.
4. Déployer, ouvrir l'URL publique.
5. Tester le parcours complet en ligne (inscription, connexion, données).

Pièges

- Oublier les variables d'env dans Vercel → app cassée en prod. À vérifier.
- Supabase : ajouter le domaine Vercel aux URL de redirection autorisées (auth).

Réflexe quota (Pro)

Déployer ne consomme pas de quota Claude ; les corrections oui.

Mon devoir

Faire tester mon app par 2 personnes et noter leurs retours et bugs.

S16

Tester la sécurité de mon app + démo finale

Règle éthique (non négociable)

Je ne teste QUE **ma propre app**, que j'ai le droit de tester. Tester l'app de quelqu'un d'autre sans autorisation écrite est **illégal**. Jamais d'exception.

Ce que j'ai appris — les failles web classiques à vérifier

- **Contrôle d'accès** (la plus fréquente) : un utilisateur A peut-il voir les données de B ? Il ne doit PAS.
- **Routes protégées** : vraiment protégées, pas juste cachées ?
- **Validation des entrées** : les formulaires acceptent-ils n'importe quoi ?
- **Fuite de secrets** : mes clés n'apparaissent-elles pas côté client ?

Outils du Jarvis (sur MA cible seulement)

- `security-auditor` (défensif, à lancer en premier)
- `web-hunter`, `api-security`, `bizlogic-hunter`, skill `webapp-exploit-hunter`

Prompt type

> « Audite la sécurité de mon app déployée (c'est mon app, j'ai l'autorisation). Commence par security-auditor, puis vérifie l'isolation des données entre deux utilisateurs, et liste les corrections par priorité. »

La faille la plus probable de mon app

Données **non isolées** entre utilisateurs → corriger les règles d'accès Supabase (RLS).

Ma démo finale (10 min)

Montrer l'app en ligne + le parcours complet + expliquer ma méthode : spec, continuité, MCP, agents, hooks, déploiement, sécurité.

La suite

Je repars avec mon Jarvis. Nouveau projet quand je veux : `/nouveau-projet`.